Manuel Utilisateur
Projet Génie Logiciel

Gr1, gl06
ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fié Victor, HURSTEL Mathis

20 Janvier 2025

Sommaire

Contents

1 Introduction **1**

2 Utilisation **1**
2.1 Première prise en main 1
2.2 Exemple de compilation basique 1
2.3 Utilisation des options 2

3 Limitations de l'implémentation **3**

4 Messages d'erreur **4**
4.1 Erreurs de lexicographie 4
4.2 Erreurs de syntaxe hors-contexte 4
4.3 Erreurs de syntaxe contextuelle 4
4.4 Erreurs d'exécution du code assembleur 4

5 Mode opératoire pour utiliser l'extension **5**
5.1 Utilisation des extensions via la ligne de commande 5
5.2 Limitations de l'extension 5

6 Annexe **5**

7 Conclusion **5**

1 Introduction

Bienvenue dans le Manuel Utilisateur de notre compilateur Deca.
Ce document vous guidera à travers les fonctionnalités, les limitations, les messages d'erreur ainsi que les extensions que notre compilateur propose.
L'objectif est de vous aider à compiler et exécuter vos programmes Deca sans difficulté.

2 Utilisation

2.1 Première prise en main

Si vous comptez profiter du compilateur sur un nouvel ordinateur (ex : machine personnelle), veuillez suivre ces quelques instructions afin de le configurer correctement.

Placez-vous à la racine du répertoire où vous avez placé le compilateur et exécutez cette commande. Elle vous permettra de créer toute la structure du compilateur sur votre ordinateur.

```
mvn compile
```

Si vous souhaitez vérifier que tout s'est bien exécuté comme prévu et que votre compilateur est fonctionnel, vous pouvez faire la commande suivante :

```
mvn verify
```

Cela permet d'exécuter la batterie de tests chargés de superviser l'installation du répertoire.

2.2 Exemple de compilation basique

Considérons un exemple simple pour illustrer le processus de compilation :
Supposons que l'on veuille compiler un fichier .deca nommé "mon_fichier.deca" sans utiliser d'option.

Compilation du fichier : Exécutez la commande suivante dans votre terminal :

```
decac mon_fichier.deca
```

Cette commande génère un fichier nommé `mon_fichier.ass`, qui est le code assembleur correspondant à votre fichier Deca. Vous pouvez le retrouver dans le même répertoire que le fichier compilé. À condition d'avoir eu recours à des options de compilation, détaillées plus bas, il est parfaitement normal que la console n'affiche aucun résultat textuel.

Execution du fichier : Exécutez la commande suivante dans votre terminal :

```
ima mon_fichier.ass
```

Cette commande exécute le programme compilé et affiche les résultats attendus dans la console. Si une erreur survient, lors de la compilation ou de l'exécution, veuillez vous reporter à la section Messages d'erreur afin de l'identifier et de la corriger.

2.3 Utilisation des options

Notre compilateur `decac` offre plusieurs options permettant d'adapter le processus de compilation en fonction de vos besoins. Voici une description détaillée de chaque option disponible et comment les utiliser:

- **-b (banner)** : Affiche une bannière contenant notre nom d'équipe. Attention, cette option ne peut être utilisée qu'isolément, sans autre option ni fichier source.

  ```
  decac -b
  ```

- **-p (parse)** : Arrête la compilation après la construction de l'arbre syntaxique et affiche une décompilation du fichier Deca. Cette option est incompatible avec l'option `-v` présentée ci-après.

  ```
  decac -p mon_fichier.deca
  ```

- **-v (verification)** : Arrête la compilation après les vérifications sémantiques, sans produire de fichier de sortie si aucune erreur n'est détectée. Cette option est incompatible avec l'ooption `-p` ci-dessus.

  ```
  decac -v mon_fichier.deca
  ```

- **-n (no check)** : Supprime certains tests d'exécution définis dans la sémantique du langage Deca.

  ```
  decac -n mon_fichier.deca
  ```

- **-r X (registers)** : Limite le nombre de registres banalisés disponibles à `R0` ... `R{X-1}`, où $4 \leq X \leq 16$.

  ```
  decac -r 8 mon_fichier.deca
  ```

- **-d (debug)** : Active les traces de débogage. Répéter cette option augmente le niveau de détail des traces affichées.

  ```
  decac -d mon_fichier.deca
  decac -d -d mon_fichier.deca
  ```

- **-P (parallel)** : Permet de compiler plusieurs fichiers en parallèle, ce qui peut accélérer le processus de compilation.

  ```
  decac -P fichier1.deca fichier2.deca fichier3.deca
  ```

- **-w (warnings)** : Active l'affichage des messages d'avertissement (*warnings*) pendant la compilation.

```
decac -w mon_fichier.deca
```

Voici quelques exemples d'utilisation combinée. Attention, le recours à plusieurs options simultanées doit respecter cet ordre d'écriture :

```
decac [[-p | -v] [-n] [-r X] [-d]* [-P] [-w] <fichier deca>...]  | [-b]
```

- Pour activer les traces de débogage et afficher les avertissements :

```
decac -d -w mon_fichier.deca
```

- Pour compiler plusieurs fichiers en parallèle et désactiver certains tests à l'exécution :

```
decac -P -n fichier1.deca fichier2.deca
```

- Pour limiter les registres à 8 et activer les avertissements :

```
decac -r 8 -w mon_fichier.deca
```

3 Limitations de l'implémentation

Dans cette section, nous vous présentons les limitations du compilateur, i.e les fonctionnalités que nous n'avons pas réussi à implémenter dans les temps impartis.

Tout d'abord, le compilateur est limité au langage Deca dit "essentiel". C'est-à-dire que le compilateur fonctionne pour la majorité du langage, mais ne supporte pas certaines opérations spécifiques :

- **Cast :** une opération qui convertit une instance d'une classe à une autre sous réserve qu'elles soient bien compatibles. Cependant, le cast est possible et se fait de façon automatique entre un entier et d'un flottant, lorsqu'on assigne ou déclare une variable de type **float** et qu'on lui attribue une valeur **int**, ou bien qu'on réalise une opération entre un **float** et un **int**.

- **instanceof :** de plus, contrairement à Java, le compilateur du langage Deca ne prend pas en charge l'opérateur **instanceof**, il est donc nécessaire d'utiliser des alternatives à cette opération :
  1) Ajouter une méthode comme `isExempleOfType()` dans chaque classe pour vérifier le type d'un objet.
  2) Utiliser un champ identifiant (`type`) dans les classes pour indiquer leur type.
  3) Exploiter le polymorphisme afin d'éviter les vérifications de type explicites et déléguer le comportement aux classes elles-mêmes.

Ces deux limitations sont des améliorations envisageables pour une deuxième version du compilateur, plus poussée. N'étant cependant pas essentielles au bon déroulement du projet, nous n'en avons pas fait une priorité, mais nous pourrions tout à fait nous pencher sur le problème avec plus de temps.

4 Messages d'erreur

Veuillez retrouver ici les différents messages d'erreur que vous serez potentiellement amenés à rencontrer lors de l'utilisation du compilateur. Ces messages d'erreur sont regroupés en quatre catégories : les erreurs de lexicographie, de syntaxe hors-contexte, de syntaxe contextuelle et d'exécution du code assembleur.
Chaque type d'erreur est détaillé dans les sous-sections suivantes.

`Message d'erreur :` chaque levée d'erreur est accompagnée d'un message de la forme :

cheminDuTest : ligne : colonne : token recognition error at : symbole qui pose problème

4.1 Erreurs de lexicographie

Les erreurs de lexicographie surviennent lorsqu'un caractère ou une séquence de caractères non reconnue par le langage est détectée (dans ce cas, par le langage Deca). Ces erreurs signalent généralement un problème lié au format ou aux caractères utilisés dans le code.

Des exemples de telles erreurs, accompagnés des messages d'erreur correspondants, sont présentés en annexe (Figures 1 à 5).

4.2 Erreurs de syntaxe hors-contexte

Les erreurs de syntaxe hors-contexte apparaissent lorsque le code source ne respecte pas les règles grammaticales définies par la grammaire du langage. Ces erreurs signalent une structure syntaxique invalide, telle qu'une parenthèse manquante ou un mot-clé mal placé. L'utilisation d'instructions et de structures DeCa est soumise à des règles strictes de sémantique, et c'est la violation d'une de ces règles qui déclenche une telle erreur. Pour les erreurs de syntaxe hors-contexte, le format est le suivant : `cheminDuTest : ligne : colonne : message d'erreur`, spécifiant l'erreur hors-contexte détectée.

4.3 Erreurs de syntaxe contextuelle

Les erreurs de syntaxe contextuelle se produisent lorsque le code source viole des contraintes de cohérence, telles que l'utilisation de variables non déclarées, des conflits de types ou des appels de méthode incorrects. Les erreurs contextuelles sont illustrées dans les Figures 8 et 9, qui présentent des exemples typiques. Par ailleurs, les descriptions des images incluent les messages d'erreur générés lors de l'exécution de ces programmes. Pour les erreurs de syntaxe contextuelle, le format est le suivant : `cheminDuTest : ligne : colonne : message d'erreur`, précisant l'erreur contextuelle détectée.

4.4 Erreurs d'exécution du code assembleur

Les erreurs d'exécution du code assembleur surviennent pendant l'exécution du programme généré. Ces erreurs incluent, par exemple, des divisions par zéro, des dépassements de mémoire ou des instructions invalides, souvent causés par des problèmes dans le code source ou au cours de la compilation. Les erreurs d'exécution du code assembleur sont illustrées dans les Figures **??** et **??**, qui présentent des exemples typiques. Par ailleurs, les descriptions des images incluent les messages d'erreur générés lors de l'exécution de ces programmes. Pour les erreurs d'exécution du code assembleur, le format est le suivant : *Description de l'erreur.*

5 Mode opératoire pour utiliser l'extension

Il est possible de compiler un langage dit étendu du langage **Deca** classique. L'extension en question est l'extension **TAB** qui permet de manipuler des tableaux statiques. La syntaxe de ces tableaux est similaire à celle des tableaux statiques en **Java**.

5.1 Utilisation des extensions via la ligne de commande

Pour utiliser l'extension, il faut utiliser l'option `-e` via la ligne de commande. Cette option vous permettra de compiler des programmes Deca comportant des tableaux. Par exemple, si dans un fichier nommé "mon_fichier.deca", vous écrivez :

```
{
    int[] x = {1,2,3};
}
```

et que vous le compilez à l'aide de la commande :

```
decac mon_fichier.deca
```

Le terminal affichera une erreur de syntaxe. Cependant, en le compilant avec la commande suivante :

```
decac -e mon_fichier.deca
```

Cela ne posera aucun problème puisque ce programme sera valide du point de vue du Deca "étendu".

5.2 Limitations de l'extension

L'extension **TAB** n'a pas de limitations significatives au niveau syntaxique ou contextuel. Pour la réaliser, nous nous sommes contentés de suivre et de copier la syntaxe utilisée en Java pour les tableaux statiques. Cependant, certaines limitations peuvent survenir dans le cadre de la réalisation de la bibliothèque de calcul matriciel.
Cette bibliothèque peut être utilisée en ajoutant la ligne suivante au début du fichier Deca à compiler :

```
#import Tab.decah
```

Cependant, cette bibliothèque n'est pas exhaustive. Nous ne pouvons pas répertorier toutes les opérations possibles avec des tableaux statiques. Pour connaître les méthodes implémentées dans la bibliothèque, nous vous invitons à consulter le fichier `Tab.decah`. Si vous souhaitez obtenir plus d'informations concernant la bibliothèque, vous pouvez consulter la documentation spécifique à l'extension. Une partie de cette documentation est entièrement dédiée à la conception de la bibliothèque de calcul matriciel.

6 Annexe

7 Conclusion

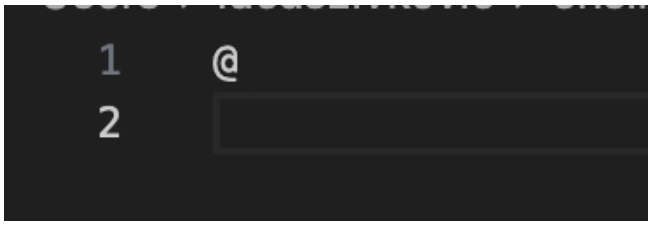


Figure 1: *$/Users/lucaszivkovic/ensimag/GL/Projet_GL/test.deca$: 1 : 0 : token recognition error at :' @'*

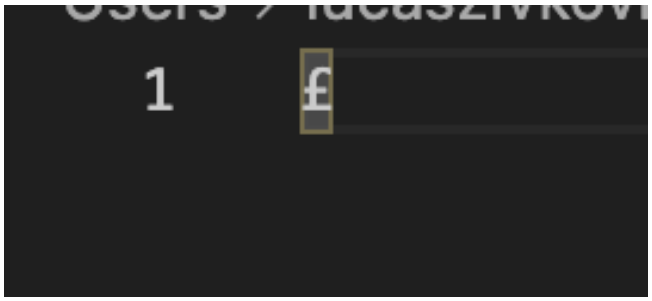


Figure 2: *$/Users/lucaszivkovic/ensimag/GL/Projet_GL/test.deca$: 1 : 0 : tokenrecognitionerrorat :' '*

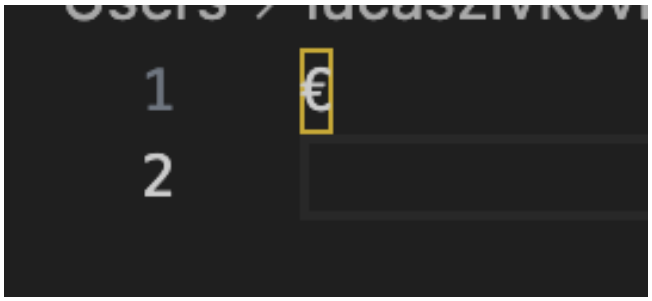


Figure 3: *$/Users/lucaszivkovic/ensimag/GL/Projet_GL/test.deca$: 1 : 0 : tokenrecognitionerrorat :' '*

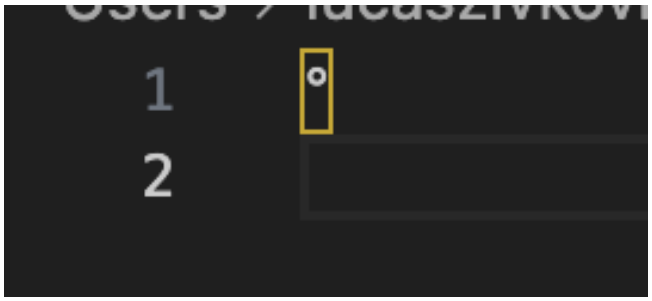


Figure 4: *$/Users/lucaszivkovic/ensimag/GL/Projet_GL/test.deca$: 1 : 0 : tokenrecognitionerrorat :' '*

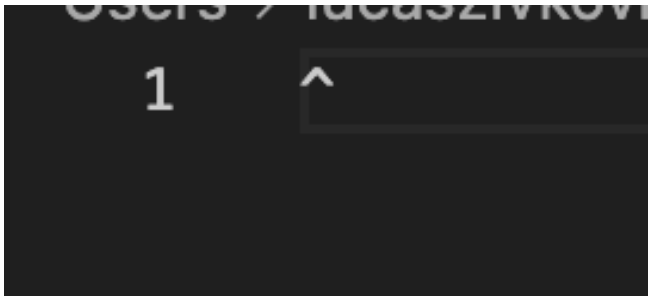


Figure 5: /Users/lucaszivkovic/ensimag/GL/Projet$_GL$/$test.deca$: 1 : 0 : $tokenrecognitionerrorat :'$

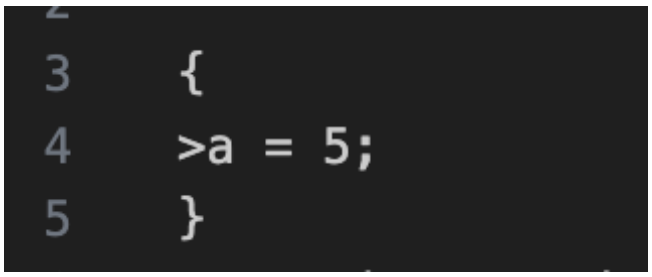


Figure 6: /Users/lucaszivkovic/ensimag/GL/Projet$_GL$/$src/test/deca/syntax/invalid/bad_synt1.deca$: 4 : 0 : $mismatchedinput' >' expecting' false',' if', ...$

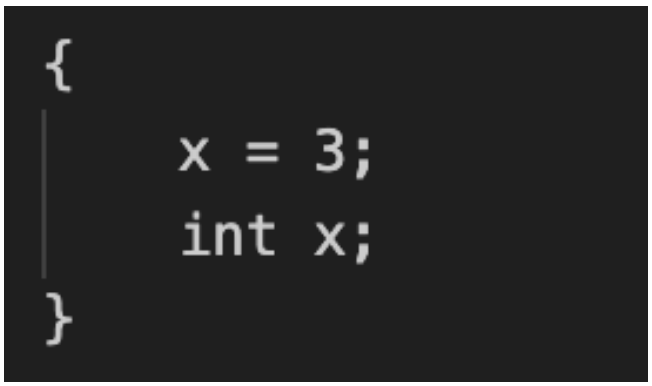


Figure 7: /Users/lucaszivkovic/ensimag/GL/Projet$_GL$/$src/test/deca/syntax/invalid/init_before_decl.deca$ 9 : 8 : $noviablealternativeatinput'x'$

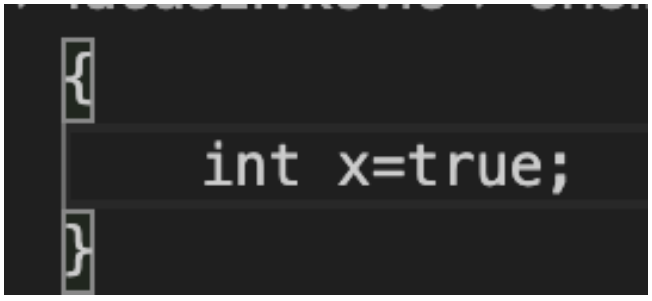


Figure 8: /Users/lucaszivkovic/ensimag/GL/Projet$_GL$/$test.deca$: 2 : 10 : $Typesincompatiblesdansl'initialisation!(attenduint), (trouvéboolean)$

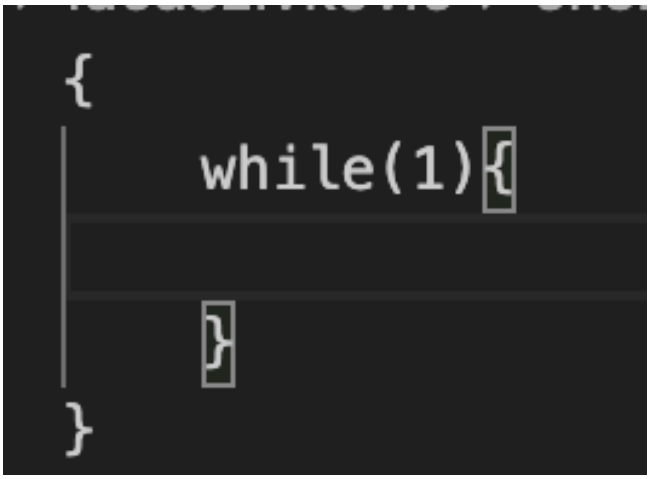


Figure 9: /Users/lucaszivkovic/ensimag/GL/Projet$_GL/test.deca$: 2 : 10 : $Laconditiondoitêtreunboolean$!

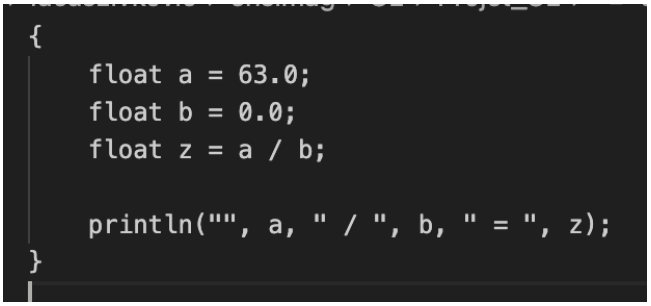


Figure 10: Erreur: Division by zero

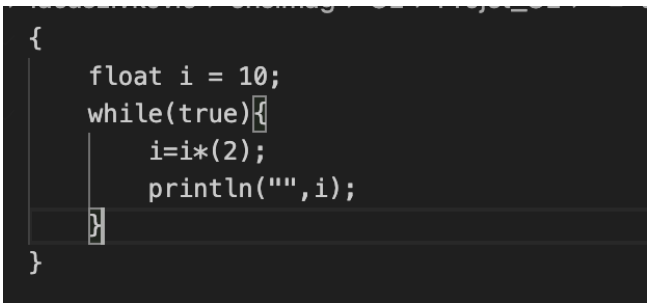


Figure 11: Erreur: Débordement en nombre flottant